Factory Feud: Adaptive Role Assignment and Threat-Aware Decision Making in microRTS

1. Motivation and Competitive Scenario

Real-time strategy games involve controlling multiple units that need to work together to achieve a common goal. These units must balance tasks like collecting resources, protecting teammates, and fighting enemies. In this project, we developed Factory Feud, a cocoa-delivery game built using the Java-based microRTS engine. Our goal was to explore whether an AI that changes unit roles based on the game situation and responds to nearby threats can perform better than an AI with fixed roles.

Factory Feud has two competing teams, each starting with four Workers and one factory. Both teams collect cocoa from a single shared resource node containing 120 cocoa and bring it back to their respective factories. Each match lasts 500 game cycles, where a game cycle represents one step in the simulation. During each cycle, all units can perform or continue actions such as moving, harvesting, or attacking. Some actions take multiple cycles to complete, so 500 cycles does not mean 500 individual moves. At the end of the match, the team that has delivered more cocoa wins. If both teams deliver the same amount, the match ends in a draw. We disabled unit production, meaning that Workers lost during combat cannot be replaced. Each Worker also has just one hit point (1 HP), so a single successful attack can eliminate it. In our visual demonstration, FixedRoleAI controls the blue team (P0), while AdaptiveRoleAI controls the pink team (P1).

2. Engine, Contributions, and Strategy Interface

microRTS provides the underlying game mechanics, including map loading, unit movement, resource collection, combat, and A* pathfinding using Manhattan distance. On top of this, we implemented Factory Feud's cocoa-delivery scenario, strategic AI controllers, threat-aware decisions, custom maps, and automated evaluation framework. We did not modify the original microRTS engine.

Game-state observations: During each decision cycle, our AI accesses microRTS's GameState to examine the positions and status of friendly and enemy Workers, remaining cocoa resources, factory locations, and player scores. This information allows the strategic layer to identify whether cocoa collection is possible, whether a Collector is threatened, and whether the factory requires additional protection. Rather than following a predetermined sequence of actions, AdaptiveRoleAI reevaluates these conditions as the game progresses.

Strategic decisions: AdaptiveRoleAI uses deterministic, priority-based rules to assign surviving Workers to Collector, Escort, Attacker, and Defender roles. Normally, the Worker nearest to the cocoa becomes Collector, another near the factory becomes Defender, one near the Collector becomes Escort, and any remaining Worker becomes Attacker. Every 10 cycles, the AI reevaluates team needs using resource availability, relative score, surviving Workers, and nearby threats. Falling behind can increase the number of Collectors, while an enemy within three Manhattan-distance tiles of a Collector can trigger additional Escort coverage. Similarly, enemies near the factory can trigger emergency reassignment toward two Defenders. Ordinary changes respect a 25-cycle per-Worker cooldown, but emergency defense and missing Collector coverage may bypass it. These rules allow the team to adjust its responsibilities as the game evolves.

Tactical decisions and engine commands: Before carrying out its assigned objective, each adaptive Worker evaluates immediate threats using factors such as enemy distance, attack range, attack timing, and nearby escape positions. If necessary, the tactical layer temporarily overrides the role's normal action with a counterattack or retreat. Otherwise, the Worker continues its assigned objective. The AI translates these decisions into high-level movement, harvesting, returning, and attack commands through microRTS's abstraction layer.

Engine execution and feedback: Our code determines each Worker's role, target, and immediate action, while microRTS handles A* navigation, action execution, collision handling, harvesting, and combat resolution. After the simulation advances, the updated GameState becomes available to the AI for subsequent decisions. This separation allows us to modify team-level strategy and threat responses without rebuilding the engine's underlying unit-control mechanisms. The controller's `getAction()` method processes the current `GameState`, while `planAssignments()` determines intended roles and `translateActions()` converts high-level orders into microRTS `PlayerAction` commands.

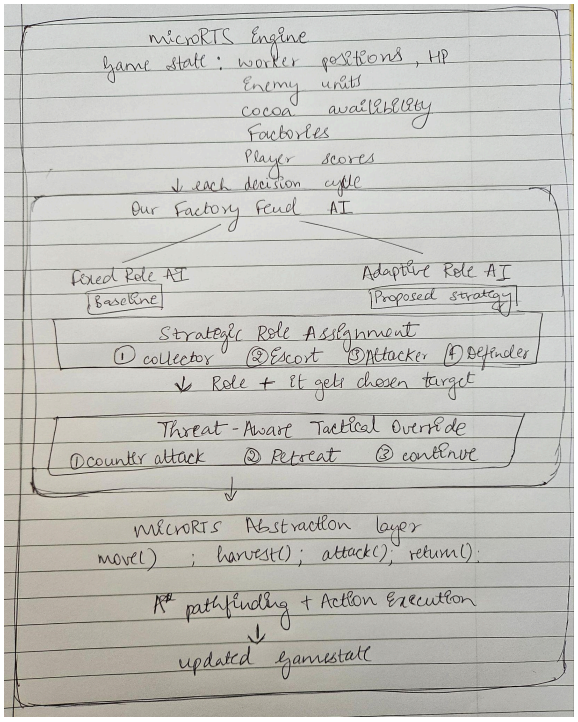


Figure 1 Strategy-to-engine interface: Factory Feud separates team-level role assignment and local threat handling from the engine's unit-command execution.

3. AI Design: Fixed Roles versus Threat-Aware Adaptation

3.1 FixedRoleAI: deterministic opponent

FixedRoleAI is our baseline opponent, where each Worker is assigned a specific role: Collector (collects cocoa and delivers it to the factory), Escort (protects the Collector), Attacker (targets enemy units), or Defender (protects friendly units and the factory). Each Worker follows its assigned role throughout the match. While it can react to nearby enemies or changing situations, it does not switch to a different role. This gives us a simple but active opponent to compare against our AdaptiveRoleAI.

3.2 AdaptiveRoleAI: Proposed Strategist

AdaptiveRoleAI uses the same four roles as FixedRoleAI but can change a Worker's role based on the current game situation. It considers resource availability, unit positions, nearby enemies, and whether collectors or the factory need protection. For example, it may switch an Attacker to an Escort when a Collector is threatened or assign a Defender to collect cocoa when more resources are needed. Cooldowns reduce unnecessary role changes under normal conditions, although emergency defensive decisions can override this restriction. This forms the strategic layer, which decides what each Worker should focus on.

In addition, we implemented threat-aware behavior to help Workers survive combat. Before following its assigned role, a Worker checks for immediate threats by considering enemy distance, attack range, damage, attack timing, and possible escape routes. Based on these conditions, it may counterattack,

retreat to a safer position, or continue its original task. This forms the tactical layer, which handles immediate combat situations without permanently changing the Worker's role.

Our approach was inspired by Barriga et al. (2017), Combining Strategic Learning and Tactical Search in Real-Time Strategy Games [1]. The authors combine high-level strategic decisions with tactical combat search. We adopted the same general principle using simpler, rule-based decisions rather than neural networks or game-tree search. By combining both layers, our AI can adjust team responsibilities while responding to immediate dangers.

4. Controlled Evaluation

We evaluated AdaptiveRoleAI against FixedRoleAI using five handcrafted maps designed to create different strategic challenges:

1. Shared Cocoa (12×12): A shared-resource arena with relatively long travel routes between the factories and cocoa.
2. Contested Center (12×12): Both teams compete for centrally located cocoa, increasing the likelihood of direct encounters.
3. Factory Pressure (12×12): Enemy activity near the factories tests whether the adaptive strategy redirects Workers toward defense.
4. Narrow Passage (12×12): Walls restrict access to the cocoa resource through a central passage, creating a chokepoint.
5. Obstacle Routes (14×12): Partial barriers create alternative routes, requiring units to navigate around obstacles while collecting and defending.

All five maps contain one shared resource node with 120 cocoa and four Workers per team, each with 1 HP. We ran two matches per map, swapping the player assignments so that AdaptiveRoleAI competed as both Player 0 and Player 1. This produced 10 matches. Each match lasted 500 game cycles and used seed 584, the ORIGINAL_FINETUNED unit configuration, and the cancel-both movement conflict policy. We measured cocoa delivered, match outcomes, surviving and eliminated Workers, adaptive role changes, counterattacks, and retreats. The experiments were executed using an automated runner that discovers the Factory Feud maps and exports match results to CSV and Markdown reports.

Swapping player sides helped account for differences in starting positions and travel distances, although it did not eliminate all possible map advantages. The simulations were deterministic, and replay verification produced identical match results. Therefore, our evaluation represents reproducible comparisons across selected maps rather than independent randomized trials.

5. Results

Map	FixedRoleAI cocoa	AdaptiveRoleAI cocoa	AdaptiveRoleAI W/D/L
Shared cocoa	2.0	3.0	2 / 0 / 0
Contested center	5.0	5.5	1 / 1 / 0
Factory Pressure	0	1.5	1 / 1 / 0
Narrow passage	5.0	5.5	1 / 1 / 0
Obstacle Routes	0.5	2.5	2 / 0 / 0

Table 1. Mean cocoa delivered across both side assignments on each map; Wins/Draws/Losses records AdaptiveRoleAI outcomes.

Across the 10 matches, AdaptiveRoleAI recorded 7 wins, 3 draws, and 0 losses. It delivered an average of 3.60 cocoa per match compared with 2.50 for FixedRoleAI, approximately 44% more. AdaptiveRoleAI lost 15 Workers overall, made 138 role changes, and executed 123 logged tactical interventions: 27 counterattacks and 96 retreats. These recorded actions confirm that both strategic role reassignment and tactical threat handling were active during gameplay.

6. Interpretation and Limitations

The results suggest that combining dynamic role assignment with threat-aware tactical decisions can improve cocoa collection performance against a fixed-role opponent. AdaptiveRoleAI won seven of the ten matches and did not lose any, indicating that the approach was effective under the tested configurations.

However, the Factory Pressure scenario highlighted a limitation of the current decision-making rules. When resource collection and factory defense were simultaneously prioritized, the AI repeatedly reassigned a Worker between the two roles. Although cooldowns were intended to reduce frequent reassignment, emergency defensive conditions could still interrupt collection behavior. Interestingly, AdaptiveRoleAI drew 0–0 as Player 1 but won 3–0 as Player 0 on the same map, highlighting the influence of starting positions. Future versions could benefit from stronger role commitment and improved prioritization between competing objectives.

Our evaluation also has several limitations. First, we tested only five handcrafted maps, so the results do not establish performance on unseen environments. Second, the experiments were deterministic, and repeating an unchanged configuration did not provide independent trials. Third, the adaptive system combines strategic role reassignment and tactical threat handling, but we did not isolate their individual effects through ablation experiments. Finally, the fixed-role baseline represents one particular opponent rather than the full range of possible RTS strategies.

7. Conclusion

Factory Feud demonstrates a functional team-level AI interface to microRTS. Our custom controller combines dynamic role assignment with local retreat and counterattack decisions, while the engine handles physical movement, harvesting, and combat. Against a deterministic fixed-role opponent, the adaptive strategy won seven matches and drew three across ten side-swapped matches on five handcrafted maps.

8. Attribution

Factory Feud was developed using the open-source microRTS Java engine [2], which provides game-state management, unit movement, A* pathfinding, harvesting, and combat mechanics. Our team implemented the custom cocoa-delivery scenario, FixedRoleAI and AdaptiveRoleAI strategies, threat-aware behavior, match controller, evaluation maps, and automated testing framework. We did not modify the original microRTS engine.

References

1. Barriga, N., Stanescu, M., & Buro, M. (2017). Combining Strategic Learning and Tactical Search in Real-Time Strategy Games. AIIDE 13(1). https://ojs.aaai.org/index.php/AIIDE/article/view/12922
2. Farama Foundation. microRTS (Java game engine). https://github.com/Farama-Foundation/MicroRTS

Appendix: Gameplay screenshots

The following screenshots show Factory Feud running on microRTS using our final AI configuration. The blue team (Player 0) is controlled by FixedRoleAI, while the pink team (Player 1) is controlled by AdaptiveRoleAI. White squares represent factories, circles represent Workers, and light-green tiles represent cocoa resources. The panel on the right displays the Workers' current roles, role changes, and tactical interventions.

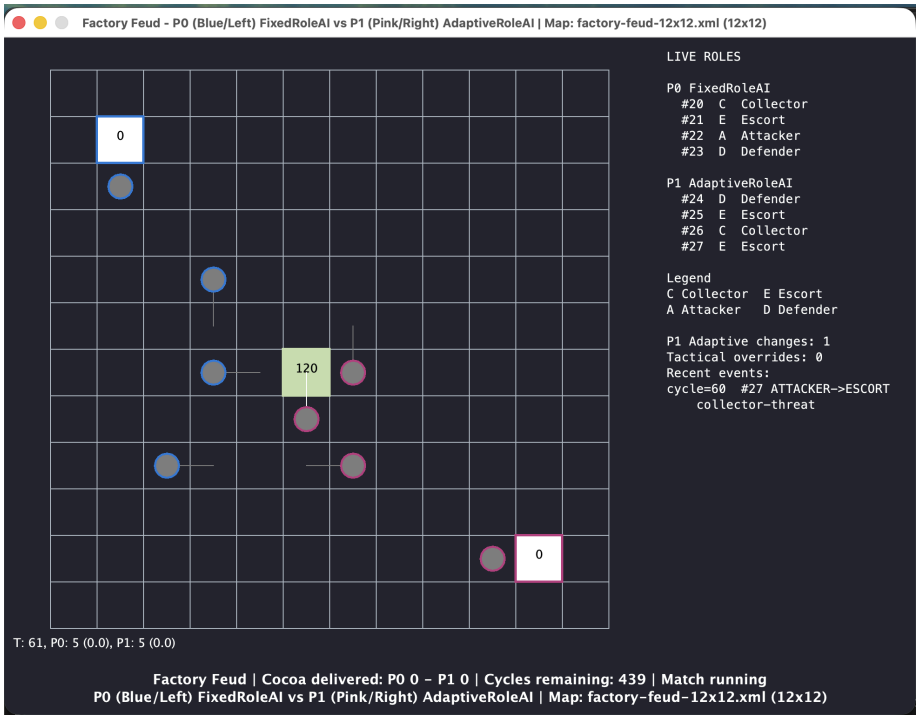


Figure A1. Factory Feud during gameplay. FixedRoleAI and AdaptiveRoleAI compete to collect cocoa from a single shared resource node. The screenshot shows the Workers' positions and their assigned roles.

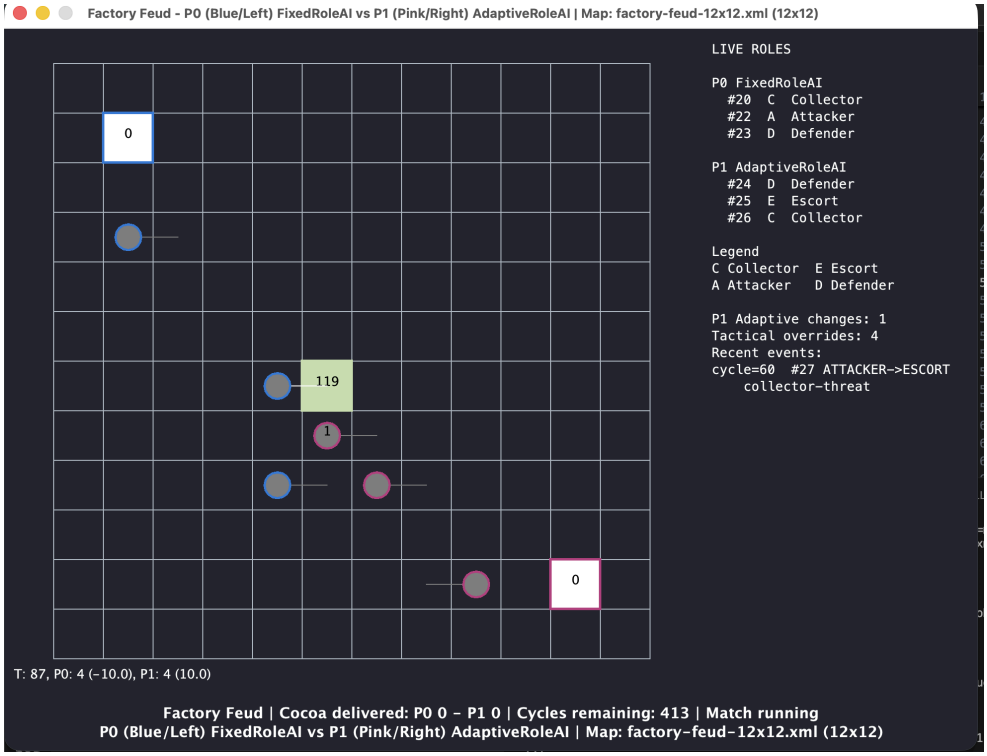


Figure A2. AdaptiveRoleAI's threat-aware behavior. The gameplay visualization shows the adaptive team's role assignments and recorded tactical interventions as Workers respond to nearby threats.

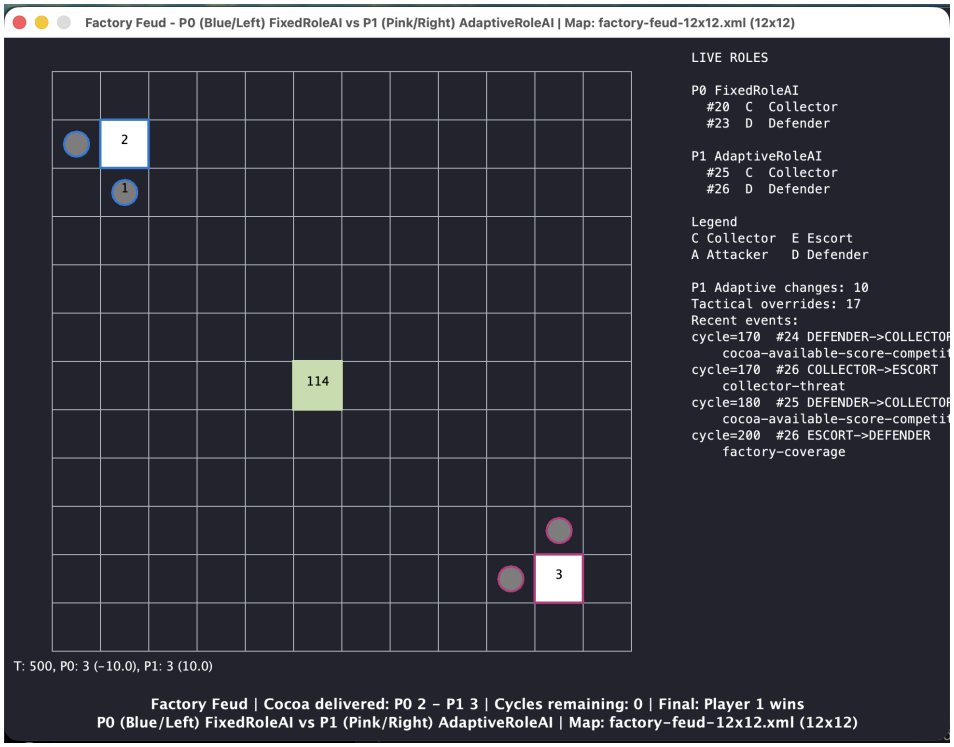


Figure A3. Final match result. The game displays the cocoa delivered by each team after 500 cycles, along with the final match outcome.

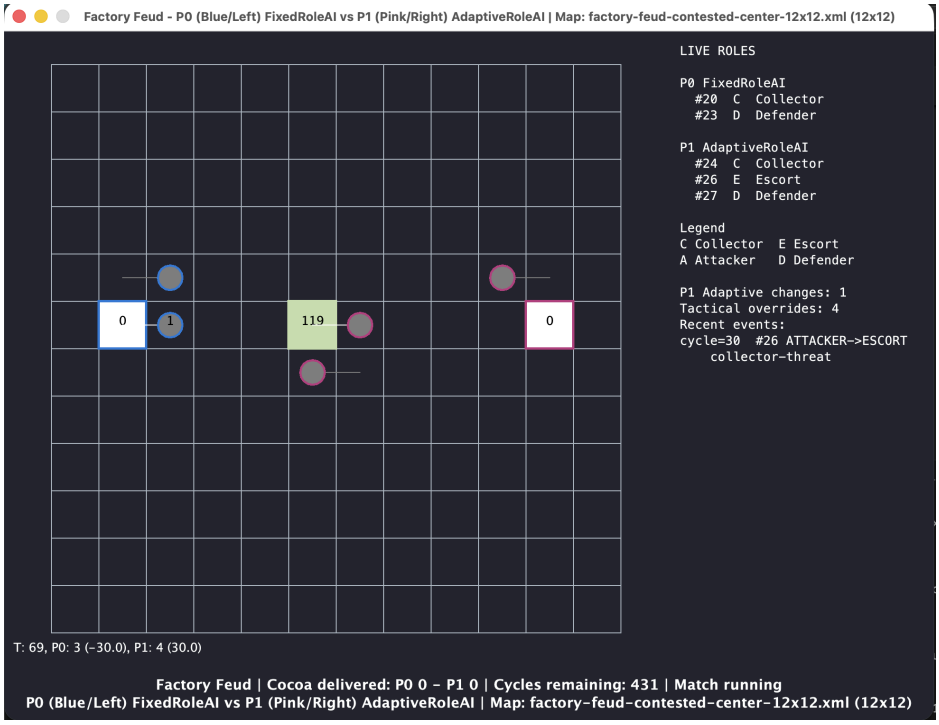


Figure A4. Contested Center (12×12). Both teams compete for cocoa near the center of the map. AdaptiveRoleAI has reassigned an Attacker to Escort duty in response to a threat to its Collector.

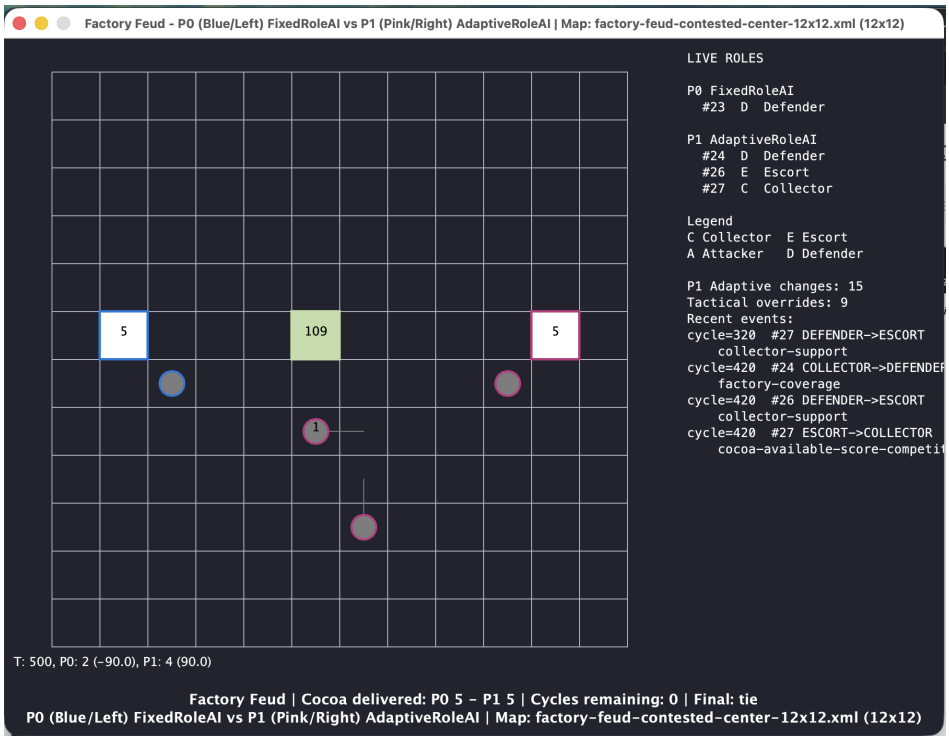


Figure A5. Contested Center (12×12): Final Match Result. The match ended in a 5–5 draw after 500 game cycles. Although both teams delivered the same amount of cocoa, AdaptiveRoleAI (pink) finished with three surviving Workers compared to FixedRoleAI's (blue) one. AdaptiveRoleAI also recorded 15 role changes and 9 tactical overrides.

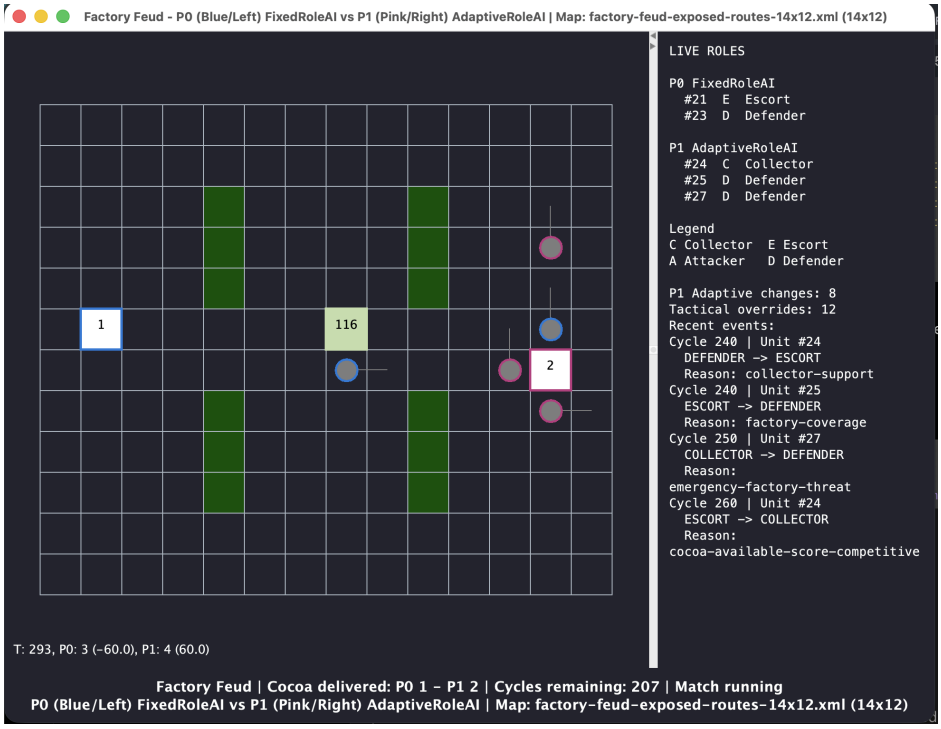


Figure A6. Obstacle Routes. The redesigned 14×12 map introduces partial barriers that create alternative routes toward the shared cocoa resource.

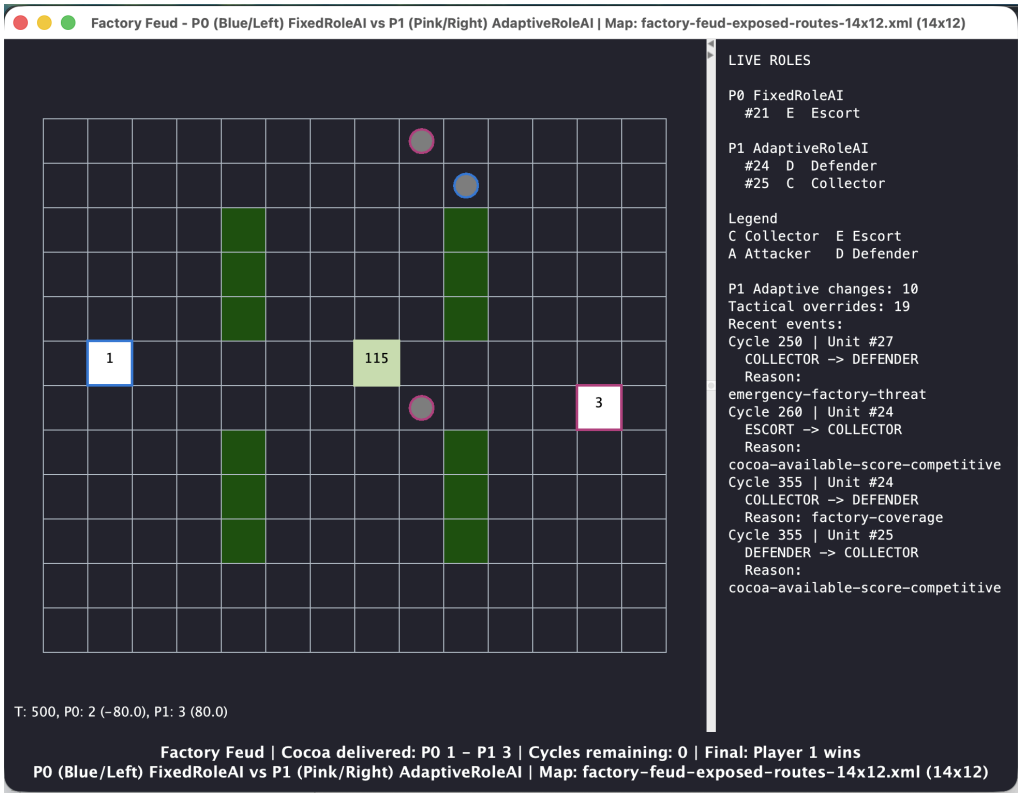


Figure A7. Obstacle Routes Final Result. AdaptiveRoleAI (pink) defeated FixedRoleAI (blue) 3–1 after 500 cycles, recording 10 role changes and 19 tactical overrides.

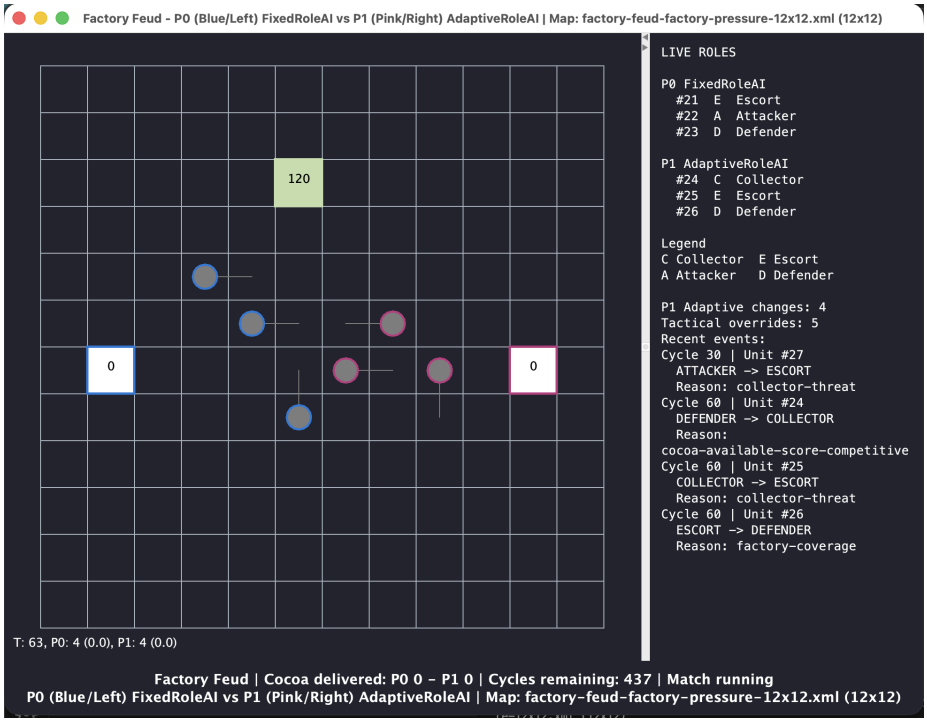


Figure A8. Factory Pressure (12×12): Early Gameplay. Both teams engage near their factories while the shared cocoa resource remains uncollected. AdaptiveRoleAI begins reassigning Workers between collection, escort, and defense roles in response to nearby threats.

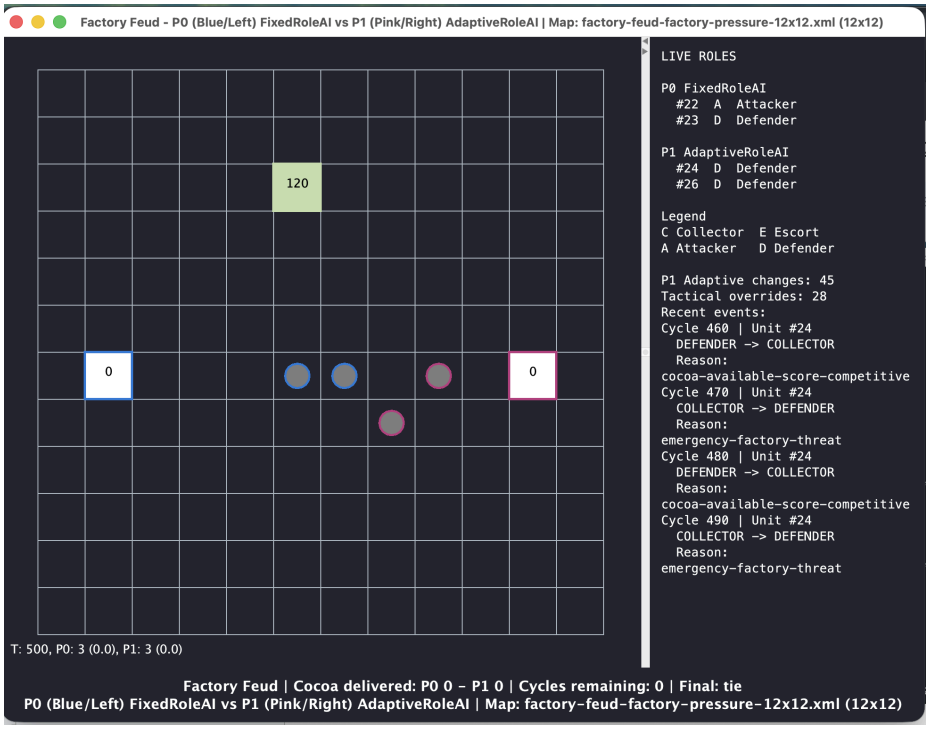


Figure A9. Factory Under Pressure (12×12): Final Match Result. *The match ended in a 0–0 draw after 500 game cycles, with neither team delivering cocoa. AdaptiveRoleAI (pink) made 45 role changes and 28 tactical overrides, repeatedly switching between collecting and defending in response to threats near its factory. This highlights a limitation of the adaptive strategy under sustained pressure.*

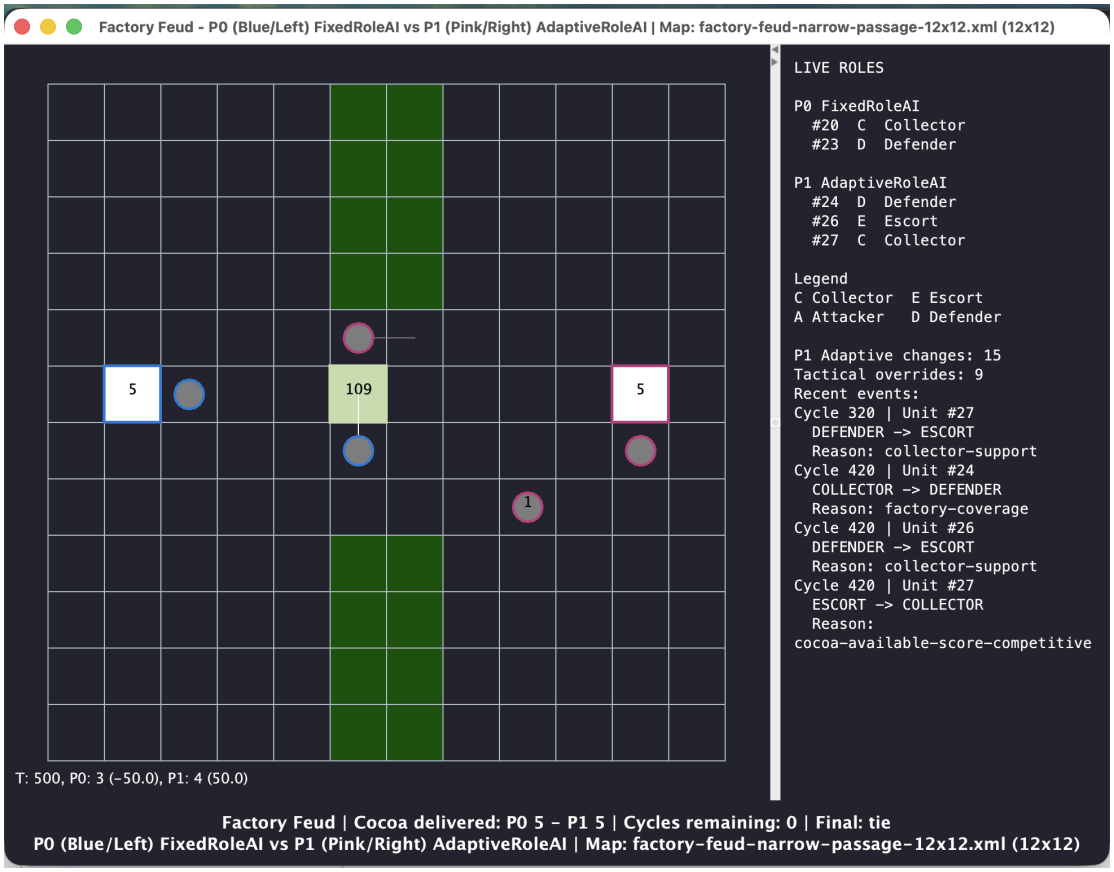


Figure A10. Narrow Passage (12×12): Final Match Result. FixedRoleAI (blue) and AdaptiveRoleAI (pink) finished in a 5–5 draw after 500 game cycles. AdaptiveRoleAI recorded 15 role changes and 9 tactical overrides while competing for resources through the central passage.